# When a Teammate Stops Moving

Robustness of Learned Robot Team Coordination to Failures Partway Through an Episode

Melinda Tran and Ellen Zheng
EC518 Robot Learning, Boston University
{meltran, ellenz}@bu.edu

***Abstract*—Robot teams trained with multi-agent reinforcement learning (MARL) are almost always evaluated with every robot working for the whole episode, but real robots stall, lose power, or get pulled from the field. We study how learned coordination holds up when a teammate fails partway through an episode, using the VMAS Navigation task and the IPPO and MAPPO implementations in BenchMARL. Preliminary experiments with teams of three and six robots show that a teammate that freezes in place lowers the success of the surviving robots, while a teammate that is removed does not. In the six robot team with a stall at step 15, about 70% of the lost success comes from survivors whose goal the stalled robot is blocking. Even after leaving those survivors out, success drops by 1.2 to 1.9 percentage points across both team sizes and both algorithms. This project asks why those survivors fail, how the effect grows when more of the team fails, and whether adding random stalls during training reduces the drop without hurting performance when nothing fails. That last step adapts the random fault injection used in recent fault tolerant MARL work to a physical stall in continuous navigation.**

## I. Introduction

Warehouse fleets, drone swarms, and field robots all share space with teammates that can break. When one robot freezes in a corridor, the others have to route around it. When one drops out entirely, the rest of the team has a little more room and one less task done. MARL policies learn to coordinate in training episodes where none of this happens, so it is not obvious how they behave once a teammate fails.

Most robustness work in MARL looks at noisy observations, model uncertainty, or teammates that act against the team [8]. A smaller line of work studies malfunctions directly. Findik et al. [5] show that methods using centralized training and decentralized execution recover slowly, if at all, when a teammate is immobilized after training has converged. They study recovery through continued training in a grid world and in a MuJoCo ant whose legs are separate agents. Mguni et al. [6] instead cause malfunctions during training, with the faulty agent taking random or adversarial actions, so the rest of the team learns to respond. They evaluate on Traffic Junction, Level Based Foraging, MPE SimpleTag, and SMACv2.

Our setting is different in a few ways. We evaluate fixed PPO policies with no retraining after the failure, which is closer to what happens on a deployed robot. The task is continuous navigation, where a stalled robot becomes a physical obstacle that never gets out of the way. We also compare a stall against removing the robot entirely, which separates the cost of losing a teammate from the cost of having a stuck one in the way. On top of that we test a simplified version of MARTA's fault injection training that uses random stalls.

This project builds on experiments Ellen Zheng started before the course. That earlier work produced the IPPO and MAPPO baseline policies for three and six robots, reference teams with one fewer robot, the single failure evaluation, and the preliminary results in Section III. Everything in the milestones table (Section VI) is new work for EC518. That includes checking the preliminary results, trajectory logging and failure mode analysis, evaluation with several failures at once, training with random stalls, and the optional communication study.

## II. Related Work

**Simulator and training library.** VMAS [1] is a vectorized 2D physics simulator written in PyTorch with a set of robot team scenarios, including the Navigation task we use. It runs many environments in parallel on a GPU, and in the authors' benchmark it was over 100 times faster than OpenAI's particle environments. BenchMARL [2] is a MARL training library built on TorchRL with standard implementations of IPPO, MAPPO, and other algorithms, plus configuration files for VMAS tasks. We use both without changing their core code.

**IPPO and MAPPO.** MAPPO [3] trains each agent's policy with PPO and a centralized critic that sees the whole team during training. IPPO [4] trains each agent with only its own value function. Both act from local observations at test time, so the centralized critic does not mean agents communicate while acting. Schroeder de Witt et al. suggest that IPPO does well partly because it copes better with an environment that changes under it [4]. A teammate that suddenly stops following its policy is that kind of change, so comparing the two algorithms is a natural place to start. Our experiments can show whether they differ, but not on their own why.

**Recovering from malfunctions.** Findik et al. [5] propose Collaborative Adaptation, which adds a relational network of weights between agents to value based learners. They immobilize an agent, or one leg of the ant, after training converges, assume the failed agent is known, adjust the relational weights, and keep training. We keep the policies fixed and measure how they respond with no extra training.

**Training with induced faults.** MARTA [6] adds a learned Switcher that decides when and which agent malfunctions during training, and an Adversary that controls what the faulty agent does. The other agents learn to respond. In an ablation on MPE SimpleTag, the authors swap the learned Switcher for a random trigger that fires with fixed probability and find the

TABLE I

PRELIMINARY ADAPTATION DROP IN PERCENTAGE POINTS AFTER A SINGLE STALL, AVERAGED OVER FAILURE STEPS 5, 15, AND 30 WITHIN EACH SEED, WITH 5 SEEDS PER ALGORITHM. DIFFERENCE IS MAPPO MINUS IPPO, AND $p$ IS FROM AN UNCORRECTED WELCH'S $t$ TEST.

| Team | IPPO | MAPPO | Difference | $p$ |
|---|---|---|---|---|
| 3 robots | 1.36 | 1.86 | 0.50 | 0.035 |
| 6 robots | 1.25 | 1.66 | 0.41 | 0.048 |

learned Switcher does much better. We use the random trigger version as our first training method, but replace MARTA's faulty actions with a physical stall, which MARTA does not model. Reproducing the learned Switcher is more than we can do in six weeks. Our random stall results give a baseline it could be compared against later.

**Communication.** HetGPPO [7] uses a graph neural network so robots in VMAS can exchange learned messages with nearby teammates. BenchMARL supports GNN models directly. If time allows, we will use this to test whether policies that rely on messages from teammates are more or less affected when one of them fails.

## III. PRELIMINARY RESULTS

**Setup.** In VMAS Navigation each robot has its own goal and a local observation, and the team is rewarded for reaching goals while avoiding collisions. Episode length follows the BenchMARL configuration for this task. We trained IPPO and MAPPO with five seeds each for 1.2M frames, on teams of three and six robots. As a reference, we also trained teams of two and five robots from scratch. Their best evaluation returns are close to those of the full teams (for example 1.081 for IPPO with six robots and 1.058 with five), though similar returns alone do not prove the two team sizes are equally hard.

**Failure evaluation.** At step 5, 15, or 30 of a test episode, one robot either stalls, staying in place as an obstacle, or is removed from the scene. Every robot takes a turn as the failed one. Nominal, stall, and removal runs for a given seed use the same held out environment seeds, and success is measured only on the surviving robots. Since some survivors may already be at their goal when a late failure happens, we will also report results that leave those survivors out.

We also report *unblocked goal success*, which leaves out survivors whose goal is blocked by the stalled robot. A goal counts as blocked when the stalled robot sits within a fixed distance of it. This removes direct obstruction of the goal but does not prove the whole route is clear. In the preliminary results, the adaptation drop for each seed is nominal survivor success minus unblocked goal success under a stall. During the project we will also compute it with nominal success restricted to the same unblocked survivors, so both sides of the subtraction cover the same robots.

**Initial observations.** MAPPO had a larger drop than IPPO in both team sizes (Table I). After a Holm correction for the two comparisons, neither difference is significant at the 0.05 level (adjusted $p = 0.070$), so we treat it as a hypothesis to test and not yet a finding.

For six robots with a stall at step 15, survivor success falls from 0.981 to 0.930 for IPPO and from 0.971 to 0.916 for MAPPO. About 70% of that drop comes from survivors with blocked goals. Removing the teammate instead gives survivor success of 0.986 and 0.977, slightly above nominal, likely because there is one fewer robot to avoid.

The drop that remains for survivors with unblocked goals, and whether training can remove it, is the focus of this project.

## IV. RESEARCH QUESTIONS

1) **Why do survivors with unblocked goals still fail?** Our guess is that most of them get stuck near the stalled robot. Both policies only ever trained with teammates that move, so a robot waiting for the frozen one to yield may wait forever. The other possibility is that survivors keep moving but make too little progress, drifting or circling somewhere else. We will log full trajectories after each failure and label every failed survivor by its distance to the stalled robot, its progress toward its goal near the end of the episode, and whether it was still moving at timeout.
2) **How does the drop grow when more of the team fails?** For the six robot policies, we will stall or remove one, two, and three robots at the same failure steps, counting a goal as blocked if any stalled robot blocks it. We expect the drop to grow faster than linearly with the number of stalled robots, because several frozen robots can form walls that are harder to route around than a single obstacle. For each seed we will compare the drop at two and three stalls with what a straight line from the single stall drop predicts, and check whether the gap between MAPPO and IPPO changes.
3) **Does training with random stalls help, and what does it cost?** We will train IPPO and MAPPO for six robots with random stalls added during training, following the random trigger MARTA uses as an ablation [6]. We expect stall training to shrink the unblocked goal drop. We will call the cost acceptable if nominal success falls by less than one percentage point. With five seeds the confidence interval on that cost will be wide, so we will report the interval and not claim the two are equal.
4) **Does learned communication change robustness?** This is a stretch goal. Using BenchMARL's GNN models in the style of HetGPPO [7], we will train policies that exchange messages with nearby teammates and run the same failure evaluation. Messages could help survivors notice and react to a failure, or they could make each robot depend more on its teammates and make a failure hurt more. We do not have a strong guess about which.

## V. APPROACH, CODE, AND DATA

**Code.** We use VMAS [1] for the environment (https://github.com/proroklab/VectorizedMultiAgentSimulator) and

BenchMARL [2] with its TorchRL backend for training (https://github.com/facebookresearch/BenchMARL). The failure evaluation is a wrapper around the VMAS Navigation scenario that already exists in our project repository. It stalls or removes a chosen robot at a chosen step and records what happens to the survivors. For question 1 we will extend it to save positions, goal distances, and failure mode measurements at every step. A stalled robot has to stay put even when other robots bump into it, and we will document how a removed robot is taken out of collisions, observations, and rewards while keeping the policy inputs valid.

**Training with stalls.** For question 3 we move the stall logic into training. At reset, each episode gets a stall with probability $p$. In those episodes one robot and one failure step within the first 30 steps are chosen at random, and that robot stays stalled for the rest of the episode. We will try $p = 0.25$ and $p = 0.5$ and compare against the existing policies trained without failures. Once a robot stalls, its transitions are left out of the policy loss, since its actions no longer affect anything. If the shared reward option is on, its goal term is also dropped from the team reward so survivors are not penalized for a goal nobody can reach. Removal is never used in training, so it serves as an unseen failure type at test time.

**Data and resources.** There is no fixed dataset. All training data comes from the simulator, and evaluation uses environment seeds that never appear in training. Training runs on GPU nodes of the BU Shared Computing Cluster. We plan five seeds per condition, which for question 3 means 2 algorithms times 2 values of $p$ times 5 seeds, or 20 runs with six robots. We will record software versions, configurations, and run times so the results can be reproduced and so we can confirm the plan fits our compute budget.

**Evaluation.** For every condition we report survivor success, unblocked goal success, collision rate, and time to goal. Time to goal always comes with success and timeout counts so failed episodes are not hidden. Nominal success is near the ceiling and differs between algorithms, so we report drops both in percentage points and relative to nominal. All comparisons use the same evaluation scenarios and the same training budget. Confidence intervals are over training seeds, and we do not treat different failure steps or failed robots within one seed as independent runs. We compare drops between separately trained conditions with Welch's $t$ test, use paired tests when the same policy faces different failures, and report effect sizes and 95% confidence intervals next to every test. Within each group of related comparisons we apply a Holm correction.

## VI. Milestones

The final report and presentation will follow the course deadlines. We expect to spend about ten hours per week each. Questions 1 to 3 are the main scope and question 4 is optional.

TABLE II
Project milestones

| Date | Milestone |
|---|---|
| Oct 28 | Revised proposal submitted. Preliminary results, metric definitions, and references checked against logs and code. |
| Nov 4 | Trajectory logging added and failure modes of unblocked goal survivors labeled for the six robot policies (Q1). Stall training wrapper written and tested. |
| Nov 11 | Evaluation with one, two, and three failed robots finished on the six robot policies (Q2). Stall training runs launched (Q3). |
| Nov 18 | All stall training runs finished (Q3). |
| Nov 25 | Stall trained policies evaluated (Q3). Communication runs started only if ahead of schedule (Q4). Fewer hours expected over Thanksgiving. |
| Dec 2 | Analysis, figures, and report draft finished. |

## VII. Expected Results and Limitations

We expect question 1 to show a few common behaviors that lead survivors to fail after a teammate stalls, and question 3 to show that some exposure to stalls during training makes policies more robust, maybe at a small cost when nothing fails. If stall training does not help, that tells us random stalls were not enough under our settings. The failure distribution, training budget, reward design, or network architecture could all be the reason, and the natural next step would be targeted fault selection like MARTA's learned Switcher.

The main limitations are a single task, simple 2D dynamics, and only five training seeds per condition. The effects are small in absolute terms because the baseline policies already succeed about 97% of the time, which is why we rely on per seed statistics and on unblocked goal success. That metric removes direct obstruction of the goal but is not a full check that the goal can be reached. A task where robots depend on each other more directly, such as VMAS Transport or Discovery, would test whether the findings carry over, but that is beyond this project.

## References


[1] M. Bettini, R. Kortvelesy, J. Blumenkamp, and A. Prorok, "VMAS: A vectorized multi-agent simulator for collective robot learning," in *Proc. 16th Int. Symp. Distributed Autonomous Robotic Systems (DARS)*, 2022. [Online]. Available: https://arxiv.org/abs/2207.03530

[2] M. Bettini, A. Prorok, and V. Moens, "BenchMARL: Benchmarking multi-agent reinforcement learning," *J. Mach. Learn. Res.*, vol. 25, no. 217, pp. 1–10, 2024. [Online]. Available: https://jmlr.org/papers/v25/23-1612.html

[3] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://arxiv.org/abs/2103.01955

[4] C. Schroeder de Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. S. Torr, M. Sun, and S. Whiteson, "Is independent learning all you need in the StarCraft multi-agent challenge?" arXiv:2011.09533, 2020.

[5] Y. Findik, H. Hasenfus, and R. Azadeh, "Collaborative adaptation for recovery from unforeseen malfunctions in discrete and continuous MARL domains," arXiv:2407.19144, 2024.

[6] D. Mguni, Y. Sun, H. Chen, W. Yang, A. Darabi, L. O. Orimoloye, and Y. Yang, "Learning robust multi-agent policies via selective adversarial fault induction," arXiv:2508.08800, Feb. 2026.

[7] M. Bettini, A. Shankar, and A. Prorok, "Heterogeneous multi-robot reinforcement learning," in *Proc. 22nd Int. Conf. Autonomous Agents and Multiagent Systems (AAMAS)*, 2023. [Online]. Available: https://arxiv.org/abs/2301.07137

[8] S. Li, J. Guo, J. Xiu, R. Xu, X. Yu, J. Wang, A. Liu, Y. Yang, and X. Liu, "Byzantine robust cooperative multi-agent reinforcement learning as a Bayesian game," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2305.12872